Lab 2 - HTML/Binding

Predaja: Četvrtak 17.4.2026.

Zadaci i bodovanje

Kriterij	Bodovi
Prompt za sub-agenta za UI/UX	1
Log da je sub-agent pozivan za UI/UX	1
Napravljen unique UX (non standard) koji radi s mock repository-ima	2
Usmeno ispitivanje razumjevanja rada s custom agentima	1

- [ ] Sav kod treba biti na GH repozitoriju, default branch
  - [ ] Kreirati custom agenta (sub-agent) za UX — definirati prompt koji opisuje kako bi UX trebao biti rađen (stil, komponente, layout principi). Osigurati da je agent instruction file commited na Git.
- [ ] Osigurati da glavni agent spawna UX sub-agenta pri generiranju UI koda — potreban je log kao dokaz
- [ ] Koristiti mock repository sa statičkim podacima iz Lab 1 - Osnove C# / LINQ (objektni model i popunjeni podaci)
  - [ ] Implementirati sve stranice za prikaz podataka (Index/lista) za svaki entitet — bez Create/Edit opcija
  - [ ] Implementirati stranice za prikaz detalja (Details) za svaki entitet
  - [ ] Implementirati specifičnu stranicu - custom home page ili slično (primjer s predavanja je bila stranica za rješavanje kviza)
  - [ ] Implementirati kompletnu navigaciju između svih stranica (izbornik, linkovi s liste na detalje, breadcrumbs)
- [ ] UX mora biti unique/non-standard (ne default Bootstrap template)
- [ ] Pripremiti se za usmeno ispitivanje organizacije UI/UX i kako promptati agente

---

Klijent-server komunikacija

Komunikacija između klijenta (primjerice, web browsera) i servera se odvija putem zahtjeva i odgovora – klijent šalje zahtjev (request), a server zaprima zahtjev i šalje odgovor (response). Zahtjev je definiran s nekoliko važnijih dijelova:

- **URL** – svaki zahtjev je naslovljen na neku URL adresu, koja se sastoji od protokola, domene, (porta), relativne putanje (relative path), URL parametara. Primjer: `https://moj.tvz.hr:443/Predavanja.php?pred=37372`

  Sastavni dijelovi URL-a:

  - Protokol (`https`)
  - Domena (`moj.tvz.hr`)
  - Port (`443`)
  - Relativna putanja (`/Predavanja.php`)
  - URL parametri (`?pred=37372`)
- **Zaglavlje** – dodatni parametri koji specificiraju primjerice jezik, format očekivanog sadržaja, autentikacijske podatke i sl.
- **POST vrijednosti** – zahtjev može sadržavati i dodatne parametre i vrijednosti
- **Tip zahtjeva** – GET, POST, PUT, DELETE

Postoji nekoliko tipova zahtjeva, istaknimo sljedeća dva:

- **GET** – zahtjev koji šalje parametre preko URL-a, i kao povratnu vrijednost dobiva (najčešće) HTML koji se prikazuje korisniku u web pregledniku
- **POST** – isto kao GET, no šalje dodatne parametre POST metodom koji mogu biti obrađeni na serveru. Najčešće se na ovaj način šalju podaci koji se popunjavaju u formi na web stranici.

Komunikaciju servera i klijenta moguće je pregledavati u posebnom dijelu developer tools alata (Network tab). Ako otvorimo stranicu koja na sebi ima formu za unos podataka, možemo vidjeti i naše podatke koji se šalju POST metodom na server (Form Data sekcija u developer toolsu).

Upoznavanje s MVC paradigmom

MVC je skraćenica za Model View Controller. Samim time lako je izolirati 3 bitna pojma:

- **View** – kombinacija HTML koda i server side naredbi. U konačnici, view se pretvara isključivo u HTML kod koji se može prikazati u internet preglednicima. Pri kreiranju tog HTML koda se koriste razne server-side naredbe koje generiraju odgovarajući HTML.
- **Model** – C# klasa koja definira podatke koji se prikazuju u odgovarajućem view-u. Ta ista klasa se u jednostavnijim projektima koristi i u raznim ORM alatima za spremanje u bazu podataka. U kompleksnijim projektima, to su posebne klase koje sadrže prilagođene podatke.
- **Controller** – C# klasa koja se sastoji od niza akcija koje se povezuju s određenim URL adresama.

Dodatni bitni pojmovi:

- **Akcija controllera** – jedan controller ima jednu ili više akcija koje kao rezultat vraćaju generirani HTML kod. Akcija controllera sadrži logiku pomoću koje kreira odgovarajuću instancu model klase, te model klasu prosljeđuje odgovarajućem view-u, koji serverskim naredbama izvlači podatke iz model klase i na temelju njih generira odgovarajući HTML kod koji se vraća kao rezultat akcije.
- **ViewModel** – dodatni pojam koji označava pomoćnu klasu koja sadrži sve prilagođene podatke koje koristi view da bi ispravno iscrtao odgovarajući HTML.

Razlika između Model klase i ViewModel klase: Model klasa sadrži samo podatke koji se spremaju u bazu, dok ViewModel može sadržavati dodatne redundantne podatke ukoliko je to potrebno. U velikoj većini osnovnih slučajeva, Model klasa je dovoljna; no u kompliciranijim scenarijima potrebno je uvesti dodatnu, ViewModel klasu.

Općenito, praksa je da se u view datoteke ne stavlja nikakva logika osim:

- Jednostavnog grananja – `if(Model.SomeProperty == true) { ... } else { .... }`
- Petlji – `foreach(var item in Model.Items) { .... }`
- TagHelper elemenata

Ukoliko se kompleksnija logika nalazi u view-u, vjerojatno je posrijedi konceptualna pogreška.

Česta je praksa da za jedan semantički pojam (primjerice, Quiz) imamo jednu Controller klasu (`QuizController.cs`), nekoliko potrebnih view model klasa (`Quiz.Models.cs` datoteka – više klasa u istoj datoteci), te nekoliko view datoteka (najčešće `Index.cshtml`, `Create.cshtml`, `Edit.cshtml`, `Details.cshtml`, po potrebi i druge). Dodatno, vrlo vjerojatno postoji i model klasa `Quiz` koja se koristi za spremanje u bazu podataka.

Tijek obrade jednog zahtjeva:

1. **Browser** → šalje zahtjev na server (URL: `/Home/DoSomething`)
2. **Server** → određuje koji Controller odrađuje zahtjev (Home), određuje koja akcija se poziva (DoSomething)
3. **Akcija controllera** → parsira i čita parametre, dohvaća podatke, kreira model objekt, prosljeđuje model view-u
4. **View** → koristi podatke iz modela za renderiranje HTML-a

Controller

Pri obradi zahtjeva se stvara instanca odgovarajuće Controller klase. Svaka controller klasa treba naslijediti iz Controller klase, čime nasljeđuje i postojeće metode. Unutar controller-a nalaze se **akcije**, a svaka akcija se sastoji od sljedećih bitnih elemenata:

- **Naziv akcije** – zapravo je naziv metode. Primjerice "Login"
- **Async/sync** – akcija može biti označena s async, međutim na samo izvođenje nema utjecaja u okruženju s malo korisnika
- **Anotacija** – anotacije pobliže opisuju ponašanje akcije, i nalaze se u [uglatim zagradama] iznad naziva metode
- **Povratni tip** – Najčešće to je `IActionResult` objekt, koji obuhvaća sve standardne povratne vrijednosti neke akcije, u pravilu objekt koji nastaje kao posljedica renderiranja view predloška
- **Parametri** – parametri koji se prenose preko URL-a ili preko forme. S obzirom na tip parametra, vrijednost se automatski konvertira (primjerice, moguće je staviti odmah int, bool ili neki drugi tip)
- **Vraćanje rezultata** – pozivom `View()` metode, koja prima dva parametra:
    - **Naziv predloška** – naziv cshtml datoteke prema kojoj se generira HTML. Ukoliko ovaj parametar nije proslijeđen, naziv view-a se dodjeljuje

automatski prema nazivu akcije controller-a

- **Model** – objekt koji sadrži podatke koje prenosimo u view. Ukoliko se ne proslijedi, prilikom generiranja HTML-a ne možemo koristiti @Model objekt

```
[HttpGet]
[AllowAnonymous]
public async Task<IActionResult> Login(string returnUrl = n
ull)
{
    // Clear the existing external cookie to ensure a clean
login process
    await HttpContext.SignOutAsync(IdentityConstants.Extern
alScheme);
    ViewData["ReturnUrl"] = returnUrl;
    return View();
}
```

View

Do sada smo jedino pomoću ViewData objekta ispisivali podatke u .cshtml datoteci, međutim to nije najoptimalniji način ispisa iz razloga što **ViewData objekt nije strongly typed**, tj., mapa (dictionary).

Puno je bolje i sigurnije koristiti tipizirani model ili view model uz odgovarajući view. Objekt koji prenosimo kao model je najčešće jednostavna C# klasa koju smo sami kreirali, ili kolekcija klasa koje smo sami kreirali. Objekt koji predstavlja model se puni u akciji controllera i proslijeđuje u view preko direktive `@model` na vrhu cshtml datoteke:

```
@model LoginViewModel
@{
    ViewData["Title"] = "Log in";
}
<h2>@ViewData["Title"]</h2>
```

Ukoliko tip modela nije prepoznat automatski, potrebno je napraviti jedno od sljedećih:

- u `_ViewImports` datoteci dodati odgovarajući namespace pomoću `@using`, ili
- koristiti `@using` direktno u view-u: `@using QuizManager.Web.Models.AccountViewModels.LoginViewModel`
- koristiti puno ime klase

Željeni podaci se ispisuju unutar HTML-a naredbom `@Model.Svojstvo`:

```
<div>
    IP: @Model.IPAddress
</div>
```

Na ovaj način, pri generiranju HTML-a će se `@Model.IPAddress` zamijeniti sa vrijednosti koja je zapisana u svojstvu IPAddress objekta kojeg smo proslijedili kao model.

Nomenklatura i konvencije

Na primjeru `HomeController` controllera i `About` metode:

- U folderu `Controllers`, nalazi se `HomeController`. Svaki kontroler završava sufixom "Controller".
- `HomeController` sadrži nekoliko akcija – promotrimo akciju `About`.
- Akcija `About` vraća `return View()`. O kojem točno view-u se radi, povezuje se preko naziva akcije. Konkretno, iz foldera `Views/Home` se pokušava locirati .cshtml datoteka istog imena kao i akcija – u ovom slučaju `About.cshtml`.
- Struktura Views odgovara strukturi kontrolera – folderi `Home`, `Manage`, `Account` su upravo nazivi kontrolera iz mape Controllers.

Sve od gore navedenog moguće je prilagoditi, ali za većinu situacija upravo ovakva osnovna postavka je dovoljna.

URL parametri akcije

U akciju controller-a moguće je proslijediti i parametre preko URL query stringa. Primjerice:

`http://localhost/Home/About` → Controller: HomeController, Action: About

```
public class HomeController : Controller
{
```

```
    public IActionResult About()
    {
        return View();
    }
}
```

Međutim, ukoliko URL promijenimo na način da proslijedimo parametar:

`http://localhost/Home/About?lang=hr` → Controller: HomeController, Action: About, Parametar: lang, vrijednost = hr

```
public class HomeController : Controller
{
    public IActionResult About(string lang)
    {
        // Vrijednost varijable lang za taj specifični URL
će biti string "hr".
        return View();
    }
}
```

URL usmjeravanja (routing)

Kako bi server znao, za neki URL, koju akciju kojeg controllera da aktivira, potrebno je specificirati pravila usmjeravanja (routes).

Pravila usmjeravanja su definirana u datoteci `Program.cs`, u korijenskoj datoteci Web projekta. Bitno je znati sljedeće:

- Jedna definicija rute se može primjeniti na više controller-a i akcija – naime, definicija rute podržava parametrizaciju naziva controllera i naziva akcije
- Rute su procesirane jedna za drugom, poštivajući redoslijed, te je aktivirana prva ruta koja zadovoljava parametrima zahtjeva

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Gornja ruta definira da se URL shema može sastojati od najviše 3 dijela (recimo `HOST/Xxx/Yyy/123`), gdje:

- iz prvog parametra (`Xxx`) se iščitava naziv odgovarajućeg Controller-a → klasa `XxxController`
- iz drugog parametra se iščitava naziv odgovarajuće akcije → `ActionResult Yyy(...) { }`
- iz trećeg dijela se izvlači parametar koji se prosljeđuje akciji → `ActionResult Yyy(int id)`, gdje parametar "id" poprima vrijednost 123. Id nije nužno cijeli broj – može biti i string.

> ⚠️ **Važna napomena o nomenklaturi**: U ASP.NET MVC nomenklatura je veoma bitna i treba poštivati određena pravila:
>
> - Controller klasa za pojam "Xyz" će se zvati `XyzController`
> - Controller klasu je obvezno staviti na istu razinu s ostalim controllerima
> - View datoteke vezane uz taj controller moraju biti u mapi `Views/Xyz/...`
> - Pri definiranju rute ne koristi se naziv `XyzController`, nego samo `Xyz`

U ovoj vježbi koristit će se isključivo jednostavni routing na način da je prva komponenta URL-a naziv Controller-a a druga komponenta naziv odgovarajuće akcije. Primjerice, poziv `Home/Contact` će izvršiti `HomeController`, akcija `Contact`.

URL rute – naredbe ActionLink i Url

Česti je slučaj da je potrebno napraviti preusmjeravanje na željenu stranicu unutar ASP.NET MVC aplikacije iz samog HTML-a ili iz JavaScript funkcije – primjerice kod izrade izbornika. Zbog načina kako su rute napravljene i održavanja istih, bilo bi izuzetno nezgodno ručno definirati linkove (`<a href="..."`), već imamo na izbor koristiti nekoliko mogućnosti:

- **@Html.ActionLink()** – kao rezultat vraća `<a>` element koji služi za redirekciju. Podržava konfiguraciju proizvoljnih html parametara, kao što su dodatni atributi i/ili css klasa, itd. Ukoliko je potrebno prenijeti parametar

(primjerice, id parametar u neku željenu akciju), potrebno je predati novi (anonimni) objekt kao parametar naziva "routeValues":

`@Html.ActionLink("Home", "Index", "Home", routeValues: new { lang = "en" })`

- **@Url.Action(...)** – kao rezultat vraća string koji predstavlja željenu rutu, generiranu na temelju controllera i akcije. Taj string se može iskoristiti unutar `<a href="...">` elementa ili unutar javascript funkcije:

  `<a href="@Url.Action("About", "Home", values: new { lang = "en" })">About</a>`

- **<a asp-*> Tag Helper** – .NET Core uvodi mogućnost definiranja vlastitih HTML elemenata, te donosi niz već ugrađenih korisnih elemenata:

  `<a asp-controller="Home" asp-action="Contact" asp-route-lang="en">Contact</a>`

Svaka od gornjih naredbi generira URL koji odgovara onima definiranima u tablici usmjerenja (Route Table).

Primjer u `_Layout.cshtml`:

```
<ul class="nav navbar-nav">
    <li>
        @Html.ActionLink("Home", "Index", "Home", routeValu
es: new { lang = "en" })
    </li>
    <li>
        <a href="@Url.Action("About", "Home", values: new {
lang = "en" })">About</a>
    </li>
    <li>
        <a asp-controller="Home" asp-action="Contact" asp-r
oute-lang="en">
            Contact
        </a>
    </li>
</ul>
```

Osnove HTML jezika

S obzirom da je znanje HTML-a usvojeno na obveznom kolegiju, ovdje je dan samo osnovni pregled bitnijih HTML elemenata.

Container elementi

- `<html>` – korijenski element stranice. Unutar njega su još `<head>` i `<body>`
- `<head>` – sadrži direktive: meta podaci o stranici, naslov, reference na skripte/stilove
- `<body>` – element koji sadrži ostale elemente za prikaz sadržaja
- `<div>` – najčešće korišteni container element koji zauzima cijelu liniju u HTML strukturi (osim ako mu je postavljeno plutanje – float). Unutar njega elementi se slažu ovisno o svojim postavkama
- `<span>` – element koji neće zauzeti cijelu liniju, već samo onu širinu koja mu je potrebna
- `<table>` – služi za tablični raspored elemenata. Koristi dodatno elemente `<th>`, `<tr>`, `<td>`

Elementi za rukovanje unešenim vrijednostima

- `<form>` – element unutar kojeg se nalaze `<input>` elementi čije vrijednosti se šalju POST metodom na server
- `<input>` – postoji nekoliko različitih tipova:
  - `<input type="text" />` – jednostavni element za unos teksta
  - `<input type="submit">` – iscrtava se u obliku gumba, na čiji pritisak se cijela forma i njene vrijednosti šalju na server
- `<select>` – vrlo sličan input elementu, no sadrži predefiniran skup vrijednosti
- `<textarea></textarea>` – kontrola za unos teksta koja podržava više linija

Twitter Bootstrap

Ili popularnije, samo bootstrap, je skup javascript i CSS biblioteka koje omogućavaju lakši razvoj web aplikacija, pri tome inicijalno postavljajući dobar dizajn i pružajući niz funkcionalnosti.

Kompletna dokumentacija s nizom primjera: http://getbootstrap.com/

Grid system

Ideja je da se cijelo korisničko sučelje podijeli u mrežu manjih dijelova, koji se dalje dijele opet u mrežu, itd. Bitno svojstvo ovakve mreže je da je (manje-više) automatski prilagodljivo veličini ekrana (mobiteli, tableti, široki ekrani) te na taj način drastično poboljšava iskustvo korisnika.

Detalji: https://getbootstrap.com/docs/5.1/layout/grid/

Modal

Vrlo često korištena komponenta za prikaz informacija u popup prozoru. Uz mogućnost otvaranja popup prozora samo korištenjem HTML-a, dodatno omogućava i proširenja korištenjem javascript funkcija.

Dokumentacija: https://getbootstrap.com/docs/5.1/components/modal/

Načela pisanja view predložaka i razor sintaksa

Razor engine je mehanizam ASP.NET MVC-a kojim se pomoću C# koda olakšava generiranje HTML koda. Razor sintaksa se sastoji od nekoliko ključnih koncepata:

- C# naredba počinje ključnim znakom `@`
- Unutar svakog view-a (kojem je proslijeđen neki model) dostupan je model objekt preko naredbe `@Model` (nakon čega slijedi svojstvo ili funkcija). Treba razlikovati definiciju modela (prva linija cshtml datoteke) koja je napisana u obliku `@model` (nakon čega slijedi tip modela)
- Kada se otvori blok vitičastim zagradama `{` i `}`, direktno unutar bloka nije potrebno koristiti znak `@`
- Ključne naredbe `@:` i `<text>` služe za ispis čistog teksta unutar bloka koda

Primjer razor view-a:

```
@model QuizManager.Web.Models.ContactSuccessViewModel
@{
    ViewBag.Title = "Contact";
}

<hgroup class="title">
```

```
    <h1>@ViewBag.Title.</h1>
    <h2>@ViewBag.Message</h2>
</hgroup>

<section class="contact-data">
    <header>
        <h3>Kontakt podaci uspješno spremljeni!</h3>
    </header>
    <div>
        IP: @Model.IPAddress
    </div>
    @if (Model.LastAccess != null)
    {
        <div>
            Last access: @Model.LastAccess
        </div>
    }
    <div>
        Browser: @Model.Browser
    </div>
    <h4>Poslani kontakt podaci</h4>
    <div>
        Ime i prezime: @Model.ContactData.Ime @Model.Contac
tData.Prezime
    </div>
    <div>
        Email: @Model.ContactData.Email
    </div>
    <div>
        <pre>@Model.ContactData.Poruka</pre>
    </div>
</section>
```

Controller se brine o tome da kreira model, napuni ga s podacima te ga proslijeđuje u View, koji onda s obzirom na podatke u modelu prezentira informacije korisniku.

Plan izrade aplikacije

Tokom narednih vježbi, napravit ćemo aplikaciju za konzultante – gdje će oni moći voditi evidenciju o svojim klijentima i sastancima s njima.

Funkcionalnosti: popis klijenata, detalji o jednom klijentu, popis sastanaka sa klijentom, detalji jednog sastanka, unos datoteki (file upload), kalendarski pregled nadolazećih sastanaka (po svim klijentima), pretraga klijenata/sastanaka.

Svaka nadolazeća vježba će se velikim dijelom naslanjati na prijašnje vježbe, tako da je potrebno razumjeti obrađeno gradivo.

Index – lista elemenata

Česta je situacija da u aplikaciji trebamo napraviti tablicu podataka. Model koji se proslijeđuje u view je kolekcija ili kompleksniji objekt koji unutar sebe ima kolekciju. Primjer – lista gradova prikazana koristeći `foreach`:

```
@model List<Vjezba.Web.Models.Mock.City>

<h2>Pregled gradova</h2>
@foreach(var city in Model)
{
    <div>
        Naziv grada: @city.Name
    </div>
}
```

Češći način prikaza je koristeći HTML `<table>` element.

Details – pregled detalja

Uz listu, prilagođeni pregled detalja se također veoma često koristi u poslovnim aplikacijama. Na Index stranici se prikazuju jednostavniji/kompaktniji podaci, dok se na stranici detalja prikazuju detaljniji podaci – u kontekstu aplikacije za evidenciju klijenata, to bi mogli biti slika, ime, prezime, adresa, sumarni podaci o sastancima, popis sastanaka, itd.

Mock repository i dependency injection

U ovoj vježbi podaci se ne dohvaćaju iz prave baze, nego iz **mock repository** klasa koje vraćaju statičke podatke. Time je moguće vrlo brzo razviti i demonstrirati aplikaciju, a kasnije bez velikih promjena zamijeniti mock implementaciju pravom bazom podataka.

Mock repository

Tipičan pristup je da za svaki glavni entitet postoji zasebna klasa koja zna vratiti podatke za taj entitet. Za Lab 2 to znači da se podaci iz objektnog modela i napunjenih instanci iz Lab 1 mogu preseliti u repozitorije kao statički ili unaprijed pripremljeni podaci.

Najčešće metode koje takav repozitorij ima su:

- `GetAll()` – vraća sve zapise potrebne za Index/lista stranicu
- `GetById(int id)` – vraća jedan konkretan zapis za Details stranicu

Primjeri naziva klasa:

- `AuthorMockRepository`
- `QuizMockRepository`

Prednost ovakvog pristupa je da controller ne mora znati *odakle* podaci dolaze. Njegov je posao samo tražiti podatke i proslijediti ih view-u.

Dependency injection

U modernim .NET aplikacijama uobičajeno je da se ovisnosti ne kreiraju ručno pomoću `new` unutar controllera, nego se registriraju u `Program.cs`, a framework ih zatim automatski prosljeđuje kroz konstruktor controllera.

Primjer registracije mock repository klasa:

```
builder.Services.AddSingleton<AuthorMockRepository>();
builder.Services.AddSingleton<QuizMockRepository>();
```

Primjer korištenja u controlleru:

```
public class AuthorController : Controller
{
    private readonly AuthorMockRepository _authorRepositor
y;
```

```
    public AuthorController(AuthorMockRepository authorRepo
sitory)
    {
        _authorRepository = authorRepository;
    }

    public IActionResult Index()
    {
        var authors = _authorRepository.GetAll();
        return View(authors);
    }
}
```

Na ovaj način controller ostaje jednostavan:

- prima ovisnosti kroz konstruktor
- dohvaća podatke iz repository klase
- šalje podatke odgovarajućem view-u

Kasnije se mock repository može zamijeniti pravim repositoryjem ili servisom bez velikih promjena u controllerima.

AI-asistirani razvoj i rad sa sub-agentima

Za Lab 2 nije dovoljno samo generirati HTML i CSS. Bitno je razumjeti **kako** se radi s agentima, kako se agentima daje kontekst i kako se kritički provjerava rezultat koji su generirali.

Preporučeni način rada

- Krenuti iterativno, korak po korak. Najprije napraviti jednostavne ekrane poput liste autora ili liste kvizova, zatim Details stranice, a tek onda specifičnu custom stranicu.
- Nakon svake zadovoljavajuće cjeline spremiti promjene na Git. Dobra praksa je stage-ati promjene s kojima ste zadovoljni prije idućeg većeg zahvata.

- Nakon veće logičke cjeline otvoriti novi chat kako bi kontekst ostao čist i stabilan.
- U kontekst eksplicitno dodati relevantne datoteke kako agent ne bi morao pogađati koje klase, view-ove i CSS datoteke treba analizirati.

Ask mode i edit mode

Važno je razumjeti razliku između načina rada agenta:

- **ask mode** – koristan za objašnjenja, analizu i pitanja, ali bez izmjene koda
- **edit/command mode** – koristi se kada agent treba mijenjati ili dodavati datoteke

Ako agent ne može mijenjati kod, vrlo često je problem upravo u tome što je odabran pogrešan način rada.

Sub-agent za UX/UI

Jedan od zahtjeva Lab 2 je da postoji specijalizirani sub-agent za UX/UI. Dobar pristup je:

- definirati jasan instruction prompt za UX agenta
- ograničiti mu alate samo na ono što mu treba za UI posao
- koristiti ga za složenije vizualne i UX dorade
- zadržati log poziva kao dokaz da je zaista korišten

Takav agent može, primjerice, dobiti zadatak da doradi layout, navigaciju, kartice, tipografiju, kontraste, stil gumba ili specifičnu custom stranicu.

Kritička provjera rezultata

AI generirani rezultat nije automatski dobar rezultat. Nakon svake veće izmjene treba provjeriti:

- je li navigacija potpuna i logična
- jesu li linkovi između lista i detalja ispravni
- je li tekst čitljiv i kontrast dovoljno dobar
- odgovara li UI zadanom stilu i izgleda li dovoljno unique/non-standard
- razumijete li C# kod, routing i vezu između controllera, repositoryja i view-a

Za usmeno ispitivanje posebno je važno da možete objasniti što agent radi, zašto je nešto generirao baš na taj način i kako biste ručno ispravili eventualnu pogrešku.

Model binding

Model binding je jedan od osnovnih koncepata MVC paradigme.

Razlike između POST i GET metode

Akcije controllera dijelimo na sljedeće dvije bitne skupine:

- **GET akcije** – obrađuju zahtjeve koji kao rezultat vraćaju podatke za pregled, i ne šalju POST parametre na server. Anotacija: `[HttpGet]`
- **POST akcije** – služe za obradu podataka koji su unešeni preko forme. Anotacija: `[HttpPost]`

Primjer GET i POST akcije za kontakt formu:

```
public ActionResult Contact()
{
    ViewBag.Message = "Your contact page.";
    return View();
}

[HttpPost]
public ActionResult Contact(FormCollection formData)
{
    //Obrada podataka
    return View("Contact");
}
```

Dvije različite akcije (iako se jednako zovu i jednak im je URL):

- Ukoliko se ne navede eksplicitno HTTP METHOD, podrazumijeva se GET (gornja akcija)
- Donja akcija će se zvati samo ukoliko se radi o POST metodi

HTML forma treba koristiti `method="post"`:

```
<h3>Pošaljite nam upit!</h3>
<form action="/Home/Contact" method="post">
    <div>
        Ime: <input type="text" name="ime" />
        Prezime: <input type="text" name="prezime" />
    </div>
</form>
```

Povezivanje vrijednosti forme i parametara akcije

Pristup 1: FormCollection (najlošiji)

```
[HttpPost]
public ActionResult Contact(FormCollection formData)
{
    var ime = formData["ime"];
    var prezime = formData["prezime"];
    return View();
}
```

Problemi:

- Velika mogućnost pogreške jer se sva imena prenose preko stringova
- Potrebno je pisati vlastiti kod za pretvorbu raznih tipova
- Posebni načini rukovanja bool vrijednostima

Pristup 2: Jednostavni parametri (bolji)

```
[HttpPost]
public ActionResult Contact(string ime, string prezime)
{
    //Obrada podataka
    return View();
}
```

Preko atributa `name` u HTML jeziku se definira naziv tog parametra u akciji. Ovo je bolje, ali i dalje: broj parametara u metodi može drastično rasti i nije se

smanjila mogućnost pogreške krivog naziva.

Pristup 3: Model binding (najbolji)

Moguće je ostaviti ASP.NET MVC-u da automatski popuni polja u objektu prema imenima HTML input polja. Ukoliko se kao parametar akcije očekuje kompleksni objekt (vlastito kreirani model), ASP.NET MVC mehanizam će pokušati kreirati instancu tog objekta, te popuniti njegova svojstva s obzirom na imena polja na formi:

```
public class ContactModel
{
    public string Ime { get; set; }
    public string Prezime { get; set; }
}
```

```
<form action="/Home/Contact" method="post">
    <div>
        Ime: <input type="text" name="ime" />
        Prezime: <input type="text" name="prezime" />
    </div>
</form>
```

```
[HttpPost]
public ActionResult Contact(ContactModel model)
{
    var ime = model.Ime;
    var prezime = model.Prezime;
    return View();
}
```

Pristup 3b: Razor EditorFor (najtipskiji)

Kako bi riješili problem mogućnosti pogreške u nazivu parametara, koristimo razor funkcionalnost koja se bazira na stablima izraza kako bi se napravilo HTML input polje na način da se specifično povezuje sa željenim poljem u modelu:

```
@model QuizManager.Web.Models.ContactModel

<section class="contact-form">
    <header>
        <h3>Pošaljite nam upit!</h3>
        <form action="/Home/Contact" method="post" class="f
orm-inline">
            <div>
                Ime: @Html.EditorFor(p => p.Ime, new
                    { htmlAttributes = new {
                        @class = "form-control",
                        placeholder = "Pretraga po nazivu"
}
                    })
                Prezime: <input type="text" name="prezime"
/>
            </div>
```

Gornji način se može koristiti samo u slučaju da polja odgovaraju direktno poljima modela, stoga je prvo potrebno razumjeti koncept korisničke kontrole/djelomičnog pogleda (partial view) kako bi se ova funkcionalnost mogla implementirati na ekran za pregled/pretragu klijenata.